

# INFORME TÉCNICO: SIMULADOR DEL PLANIFICADOR DE CPU PARA EL SISTEMA SIGET

Valeria Villa Rua

<https://github.com/valeriavilla906/Tarea.git>

**Materia:** Sistemas Operativos

La gestión de procesos en un sistema operativo representa una de las funciones más críticas dentro de la arquitectura de software, ya que es la encargada de administrar la asignación del procesador a las distintas tareas concurrentes. En el contexto particular del Sistema Integrado de Gestión de Tráfico (SIGET), el motor de procesamiento se enfrenta a un desafío dual: por un lado, debe procesar de forma continua flujos masivos de datos provenientes de sensores y cámaras viales; por otro lado, debe reaccionar de manera inmediata ante alertas críticas que amenazan la movilidad y la seguridad urbana, tales como la semaforización de emergencia. Para simular y analizar el comportamiento operativo de este entorno, se diseñó e implementó un simulador de CPU en lenguaje C++ que modela el ciclo de vida completo de las tareas y compara el desempeño de los algoritmos de planificación Round-Robin y Prioridades.

El diseño del simulador se fundamenta en el paradigma de programación orientada a objetos mediante la definición de la clase `Proceso`. Esta clase actúa como el bloque de control del proceso (PCB) en un sistema operativo real, encapsulando tanto los atributos fijos de cada tarea como sus variables dinámicas de estado. Entre los atributos principales configurados para el SIGET se encuentran el identificador único (`id`), el nombre descriptivo de la tarea, el tiempo estimado de irrupción en CPU (`tiempoIrrupcion`), el nivel de urgencia (`prioridadAlerta`) y el tamaño del volumen de información a procesar (`tamanoDatosMB`). Asimismo, la clase gestiona el tiempo restante de ejecución mediante `tiempoRestante` y calcula progresivamente dos métricas fundamentales de rendimiento: el tiempo de espera, que contabiliza los segundos que la tarea permanece en la cola sin uso de la CPU, y el tiempo de retorno, que mide la duración total transcurrida desde que la tarea ingresa al sistema hasta que finaliza por completo.

Para representar visualmente la dinámica del procesador, el simulador hace uso de una enumeración fuertemente tipada denominada `enum class EstadoProceso`. Esta estructura permite modelar explícitamente las transiciones entre los cinco estados estándar definidos en la teoría de sistemas operativos: Nuevo, Listo, En Ejecución, Bloqueado y Terminado. A través del método `cambiarEstado()`, el programa imprime en la consola del sistema cada cambio de fase en tiempo real (por ejemplo, mostrando la transición de *Listo* a *En Ejecución*), lo que proporciona una trazabilidad precisa del flujo de control. Además,

haciendo uso de la librería `<thread>` y el espacio de nombres `std::this_thread::sleep_for`, se incluye la función `pausar()` para simular el paso del tiempo de procesamiento en el hardware, logrando una simulación auténtica y comprensible.

El primer algoritmo evaluado en la simulación fue el esquema Round-Robin con un quantum asignado de dos segundos. En esta implementación, las tareas se ordenan en una cola FIFO (`std::queue`) en estado *Listo*. El planificador toma el proceso al frente de la cola, conmuta su estado a *En Ejecución* y le concede acceso a la CPU durante un intervalo igual al quantum o al tiempo restante de la tarea. Al restar el tiempo consumido, el programa actualiza de forma simultánea el tiempo de espera de los demás procesos que permanecen en la cola. Un aspecto relevante del código es la simulación de eventos de entrada y salida: cuando a un proceso le queda un tiempo restante múltiplo de tres, se invoca una transición temporal al estado *Bloqueado* para representar la latencia de transferencia de datos en la red del SIGET, antes de regresar a la cola de listos. Este esquema demostró ser sumamente equitativo para tareas rutinarias y de gran volumen, como el conteo vehicular masivo, evitando la monopolización del procesador; no obstante, genera cierta latencia para tareas urgentes al depender de su turno en la cola.

Por otro lado, se implementó la función `ejecutarPlanificacionPorPrioridades` para modelar la atención inmediata a contingencias viales. En este enfoque, el algoritmo inspecciona iterativamente el conjunto de tareas activas y selecciona aquella que posee el menor valor entero en su atributo de prioridad, asumiendo que el valor cero representa la máxima urgencia de alerta. Al ejecutar este algoritmo sobre el conjunto de pruebas del SIGET, la tarea "Semaforización de Emergencia" (PID 101, Prioridad 0) toma de inmediato el control de la CPU, ejecutándose hasta su culminación sin sufrir interrupciones por parte de tareas de menor prioridad o de mayor tamaño de datos. Este comportamiento garantiza que las emergencias obtengan un tiempo de espera inicial nulo, optimizando la latencia de respuesta del sistema en situaciones críticas.

Finalmente, el programa concluye su ejecución invocando la función `mostrarResumenMetricas()`, la cual genera una tabla formateada mediante la librería `<iomanip>` que detalla el PID, nombre, tiempo de irrupción, tiempo de espera acumulado y tiempo de retorno de cada proceso, calculando además los promedios globales. Los resultados comparativos confirman que mientras la planificación por Prioridades es indispensable para la atención inmediata de emergencias viales, el algoritmo Round-Robin ofrece el mejor equilibrio para el procesamiento distribuido de datos de rutina. En consecuencia, se concluye que el diseño ideal para la arquitectura del SIGET debe ser un modelo híbrido basado en Colas Multinivel Retroalimentadas, combinando la atención prioritaria de alertas con el reparto equitativo de tiempo para el flujo continuo de tráfico.